

Type Theory and Mathematical Logic

PHI 108

Aditya Pahuja

October 29, 2024

Set theory

One modern foundational system of math is *set theory*, which defines all mathematical objects in terms of sets, which, informally, are collections of elements. For example, $\{1, 0, 8\}$ is a set whose elements are 1, 0, 8.

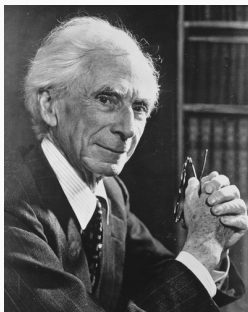


Figure 1: Bertrand Russell, a pioneer of mathematical logic.

Set theory notation

Two pieces of notation:

- To describe a set, we use an expression like

$$S = \{x : x \text{ has property } P\},$$

which says “ S is the set of elements x that have property P .”

For example

$$\{x : x \text{ is a positive integer smaller than } 3\} = \{1, 2\}.$$

- If S is a set, then we write $x \in S$ if x is an element of S and $x \notin S$ if x isn't an element of S . Thus $1 \in \{1, 0, 8\}$ and $2 \notin \{1, 0, 8\}$.

Russell's paradox

- Early formulations of set theory (now called *naive set theory*) did not place any restrictions on the property P .
- This led to *Russell's paradox*, named after Bertrand Russell: consider the set

$$R = \{S : S \notin S\}.$$

- On one hand, if $R \notin R$, then R has the defining property of the set, so R must be an element of R .
- On the other hand, if $R \in R$, then R does not have the defining property of the set, so R cannot be an element of R .

The birth of type theory

Since the existence of R led to a contradiction, mathematicians had two responses:

- Some, like Ernst Zermelo, patched set theory to create modern set theory.
- Russell created a version of what we now call *type theory*.

Over time these solutions embedded themselves in academia and in culture, such that many of them are now taken to be common sense. But what looks like common sense today was in the past not so common, and not seen as obviously sensible. More often than not, somebody somewhere had to struggle for it. (CPT 15)

Basics of type theory: types

In type theory, every object has an inherent *type*.

- To declare that an object x has a type A , we write $x : A$.
- For example, we say that natural numbers have type $\mathbb{N}$, so $x : \mathbb{N}$ means that x is a natural number.

The main idea is that objects are inseparable from their types: one cannot refer to an object without assigning it a type.

But for the purpose of reasoning as clearly and as systematically as possible, it is important to use our words very carefully. This usually means avoiding metaphors, symbols, rhetorical questions, weasel words, euphemisms, tangents, equivocations, and 'double speak'. When building a case for why something is true, or something else is not true, etc., it is important to say exactly what you mean, and eliminate ambiguities as much as possible. (CPT 80)

Basics of type theory: functions

We can also talk about functions between types. Recall that a function f with *domain* A and *codomain* B is a rule for transforming elements of A into elements of B .

- We say that f has type $A \rightarrow B$.

Basics of type theory: more abstract types

To create new types, we must specify a few pieces of information:

- A *formation* rule, which describes the constituents of the new type.
- An *introduction* rule, which describes how to construct objects of the new type.
- An *elimination* rule, which defines functions that help extract information about objects of the type.
- A *computation* rule, which describes what the functions from the elimination rule output.
- Optionally, a *uniqueness* rule, which essentially says that objects of a type are uniquely determined by the results of elimination rules applied to it.

The punchline

The kicker is that type theory can function as a system of logic with the main principle that *every proposition is a type*.

By following the guidelines noted above . . . , we can craft the kinds of sentences that can be used to build arguments. Such sentences are called propositions; or they are also sometimes called statements and claims. A proposition is a simple sentence that has just one meaning, for it expresses one thought according to the rules of grammar in the language in which it is expressed. (CPT 82)

How do we represent truth?

- A proposition is true if we can construct an object of the corresponding type.
- Implication corresponds naturally to functions!

What is truth? The origin of the English word truth is in Anglo-Saxon words like getriewe, treow, and troth, words relating to faithfulness, fidelity, honesty, promise keeping, and loyalty, especially in relation to an important undertaking. Truth, in the kind of logic we've been discussing here, is a property of propositions. (CPT 84)

Logical connectives: Conjunction

- Formation: Given propositions P and Q , we form the type $P \wedge Q$.
- Introduction: Given $p_1 : P$ and $q_1 : Q$, we can construct an object $(p_1, q_1) : P \wedge Q$.
- Elimination: Given $(p_1, q_1) : P \wedge Q$, we have functions $L(p_1, q_1) : P$ and $R(p_1, q_1) : Q$.
- Computation: $L(p_1, q_1) = p_1 : P$ and $R(p_1, q_1) = q_1 : Q$.
- Uniqueness: Given $z : P \wedge Q$, $z = (L(z), R(z))$.

Disjunctions are significantly more complicated and will be left as reading for the interested viewer.

Logical connectives: Equivalence

- Formation: Given propositions $P \rightarrow Q$ and $Q \rightarrow P$, we form the type $P \iff Q$.
- Introduction: Given $f_1 : P \rightarrow Q$ and $f_2 : Q \rightarrow P$, we can produce an object $(f_1, f_2) : P \iff Q$.
- Elimination: Given $(f_1, f_2) : P \iff Q$, we have functions $L(f_1, f_2) : P \rightarrow Q$ and $R(f_1, f_2) : Q \rightarrow P$.
- Computation: $L(f_1, f_2) = f_1 : P \rightarrow Q$ and $R(f_1, f_2) = f_2 : Q \rightarrow P$.
- Uniqueness: Given $z : P \iff Q$, $f = (L(f), R(f))$.

Logical connectives: Negation

To talk about falseness, we could describe a construction of $\neg P$ given P , but there is a subtle trick to avoid this!

- We define a type `False` with no objects (thus it is not true). `False` only has a (trivial) formation rule and an elimination rule that essentially produces an object of any type.
- We think of $\neg P$ as shorthand for the type $P \rightarrow \text{False}$ since, if P is true, then we can find an object of type `False`, which means that P cannot be true.

An example

Modus tollens says that if $P \rightarrow Q$ and $\neg Q$, then $\neg P$. In type theory, this means we have to construct an object of type $\neg P$ given objects of type $P \rightarrow Q$ and $\neg Q$.

- Let $f_1 : P \rightarrow Q$ and $f_2 : Q \rightarrow \text{False}$, recalling that $\neg Q$ is shorthand for $Q \rightarrow \text{False}$. The composed function $f_2 \circ f_1$ is then a function $P \rightarrow \text{False}$, so $f_2 \circ f_1 : \neg P$.

A fun exercise: prove that $P \wedge Q \iff Q \wedge P$.

The validity of an argument is determined not by what it says, but by its form. This means that when we assess the validity of an argument, we assume that the premises are true. . . . When we test for an argument's validity, we ignore all questions about whether the propositions are true or false. Instead, we only look at the way the propositions are arranged in relation to each other, and whether they conform to a correct logical structure. (CPT 95)

Many modern programming languages are *typed*, meaning all variables and functions have to have a type!

```
#src/main/java/com/scaler/application/Department.java

package com.scaler.application;

import java.util.Date;

public class Department
{
    private String departmentName;
    private Date startingDate;
    public Department(String departmentName)
    {
        this.departmentName = departmentName;
        this.startingDate = new Date();
    }

    public String getDepartmentName()
    {
        return this.departmentName;
    }

    public Date getStartingDate()
    {
        return this.startingDate;
    }
}
```

Figure 2: Some code in Java.

Proof assistants

Proof assistants are software tools that aid in the creation of completely rigorous mathematical proofs. Some, like Lean 4, implement a version of type theory called *dependent type theory*.

```
def AndComm : p ∧ q ↔ q ∧ p :=  
  Iff.intro  
    (fun h : p ∧ q =>  
      show q ∧ p from And.intro h.right h.left)  
    (fun h : q ∧ p =>  
      show p ∧ q from And.intro h.right h.left)
```

Figure 3: A proof that $P \wedge Q \iff Q \wedge P$ in Lean 4.

Type theory can be used to represent semantic structures in linguistics. One way to do this:

- e and t are basic types for individuals and truth values.
- If a and b are types then so is $\langle a, b \rangle$, which is the type of functions from entities of type a to entities of type b .
- For example, in the sentence “Bill is blond,” “Bill” has type e and “blond” has type $\langle e, t \rangle$ (and the sentence has type t).

Further questions

- Why is mathematical formalism important?
- Are there paradoxes in versions of type theory similar to Russell's paradox in (naive) set theory?
- How does type theory relate to other foundational systems of math and logic?

Links!

- [Propositional logic as a dependent type theory](#)
- [Theorem Proving in Lean 4](#)
- [Type theory in linguistics](#)